

Step 3 - Design Deep Dive

In the high-level design, we discussed:

- Data gathering service
- Query service

The previous design works for small datasets, but it is not efficient for large-scale systems. In this section, we improve the design by exploring:

- Trie data structure
- Data gathering service
- Query service
- Scaling the storage
- Trie operations

Trie Data Structure

Using a relational database to fetch top 5 search queries is inefficient because:

- Prefix searches are expensive
- Sorting large datasets is slow
- Database becomes a bottleneck at high scale

To solve this problem, we use a **Trie (Prefix Tree)**.

What is a Trie?

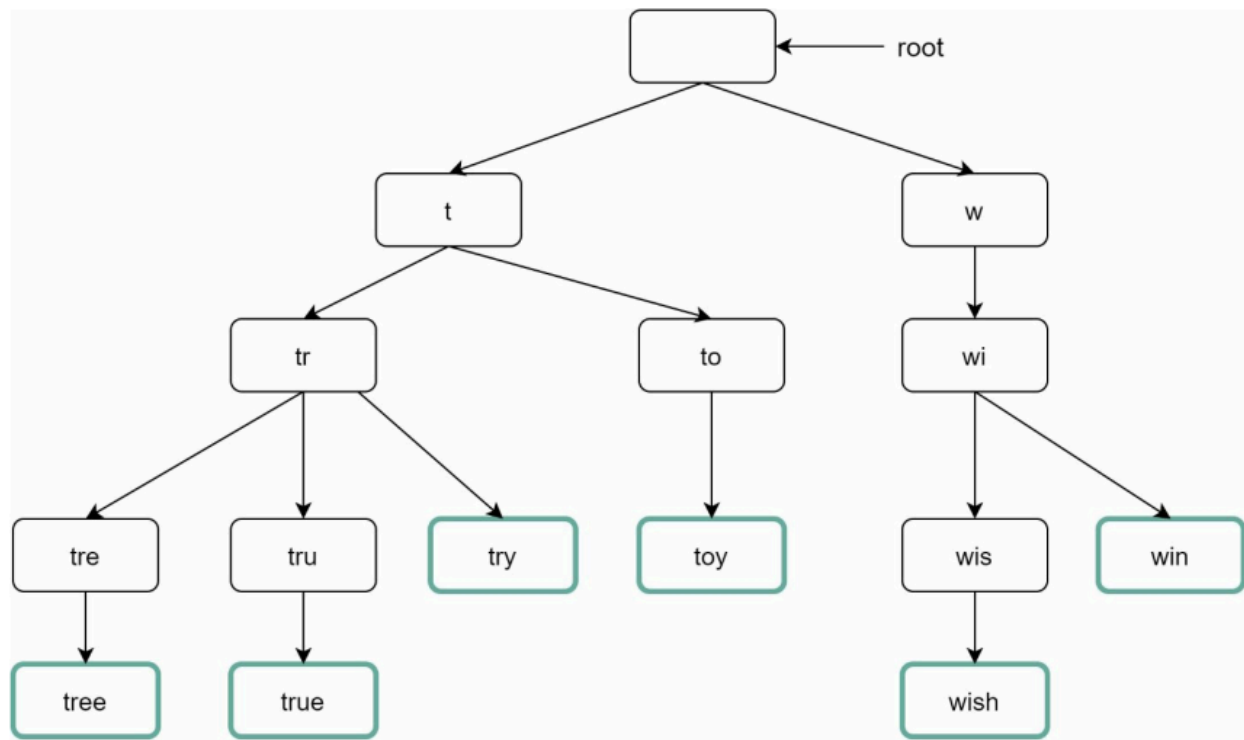
A trie is a tree-like data structure optimized for string retrieval operations.

The word “trie” comes from the word **retrieval**.

Properties of Trie

- Root node represents an empty string.
- Each node stores one character.
- Each node can have up to 26 children (for lowercase English characters).
- Each path from root to a node forms a prefix or complete word.
- Complete search queries are marked as valid words.

Example Trie



Assume the following search queries exist:

None

tree

try

true

toy

wish

win

Figure shows the trie built from these search queries.

Complete words are highlighted with thicker borders.

Frequency Information in Trie

A basic trie only stores characters.

However, autocomplete systems must return:

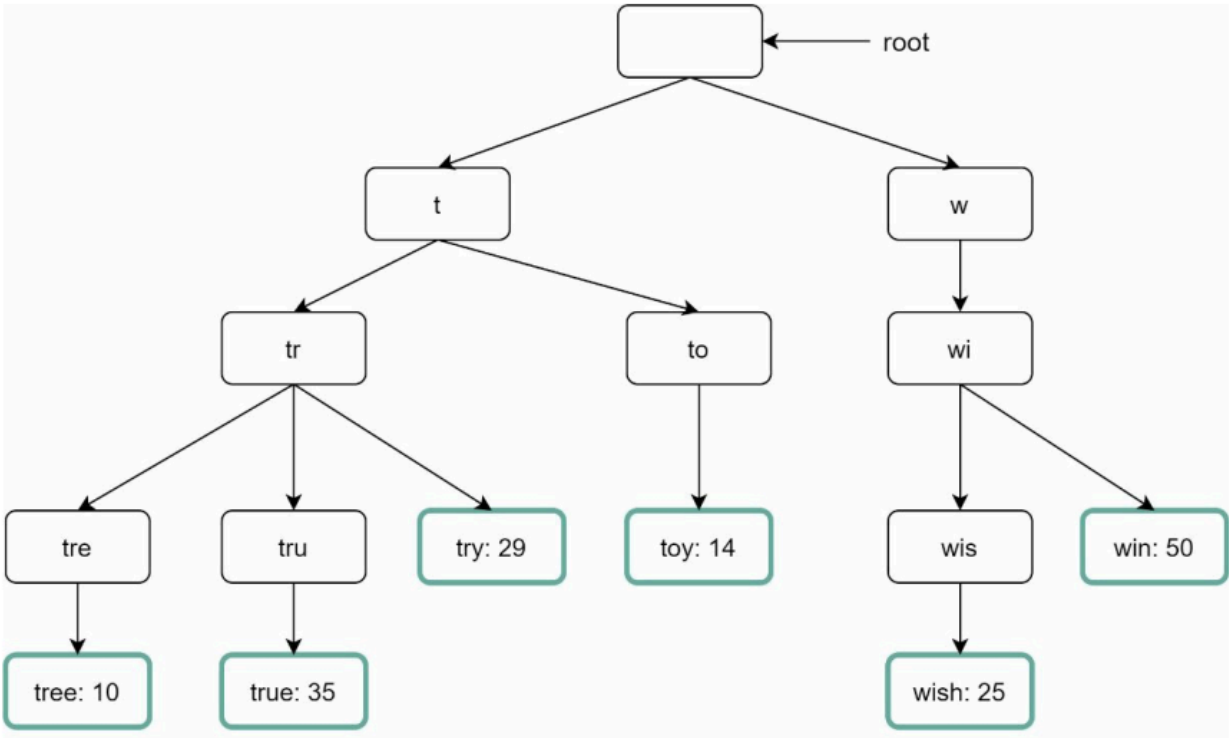
- Most popular queries
- Sorted by frequency

Therefore, frequency information is added to trie nodes.

Example Frequency Table

Query	Frequency
tree	10
try	29

true	35
toy	14
wish	25
win	50



After adding frequency information, the trie structure becomes the updated trie shown in **Figure**.

Autocomplete Search Algorithm

Define:

- p = length of prefix
- n = total trie nodes
- c = number of children under a node

To retrieve top k search queries:

Step 1: Find Prefix Node

Search the trie for the prefix.

Example:

None

tr

Time complexity:

None

$O(p)$

Step 2: Traverse Subtree

Traverse all valid child nodes under the prefix node.

A node is valid if it forms a complete query.

Time complexity:

None

$O(c)$

Step 3: Sort Results

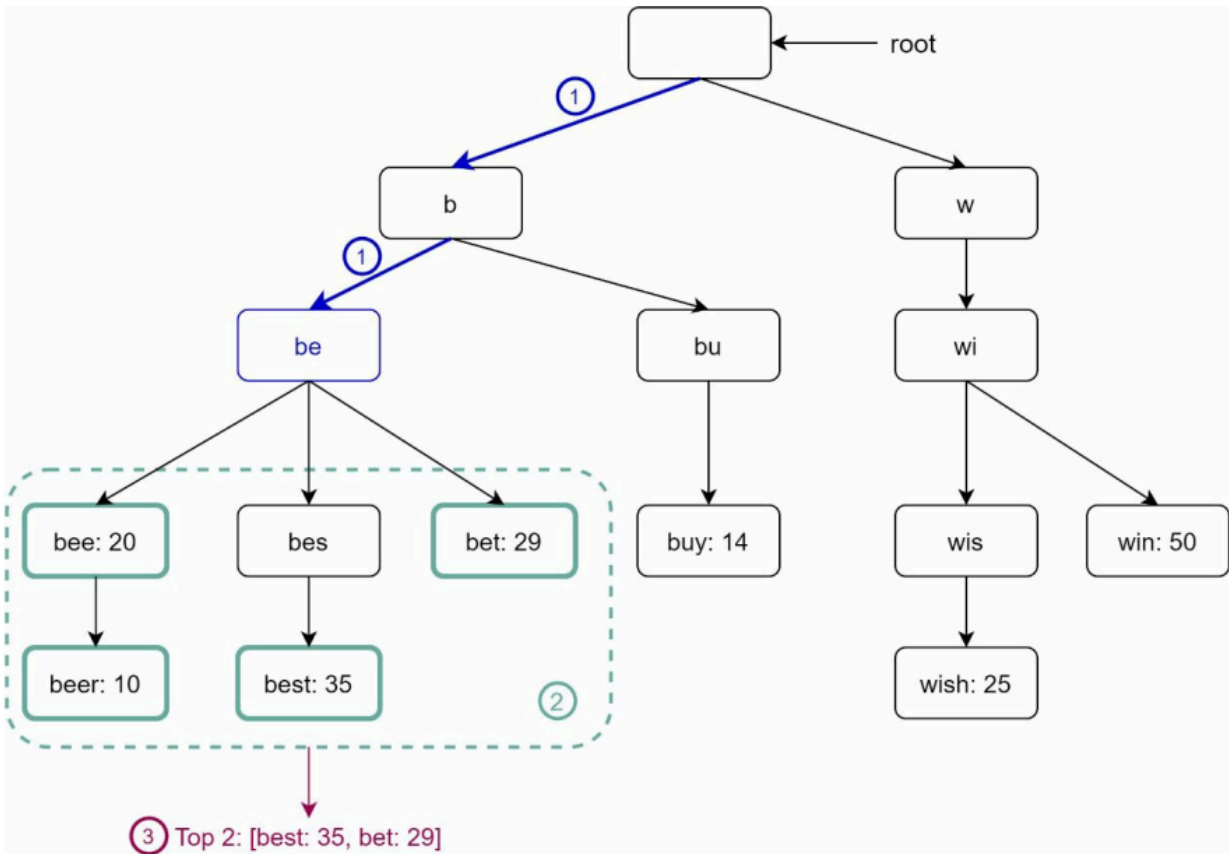
Sort valid queries by frequency and return top k.

Time complexity:

None

$O(c \log c)$

Example



Assume:

- Prefix = "tr"
- k = 2

Step 1

Find prefix node "tr".

Step 2

Traverse subtree and get valid queries:

- tree : 10
- true : 35
- try : 29

Step 3

Sort by frequency:

Query	Frequency
true	35
try	29
tree	10

Top 2 results:

- true
- try

Time Complexity

Total complexity:

None

$$O(p) + O(c) + O(c \log c)$$

This approach becomes slow when:

- Trie is very large
- Prefix subtree contains many nodes

Therefore, optimizations are needed.

Optimization 1: Limit Prefix Length

Users rarely type extremely long queries.

Assume:

None

Maximum prefix length = 50

Then:

- p becomes a small constant.
- Prefix lookup becomes:

None

$$O(1)$$

instead of:

None

$$O(p)$$

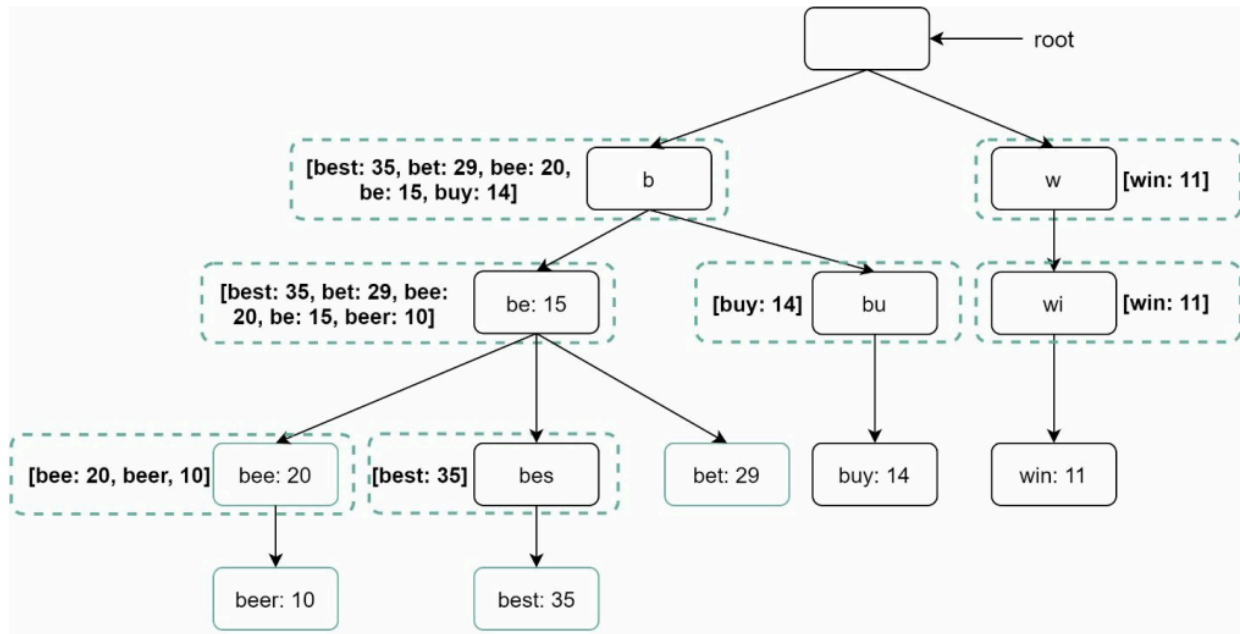
Optimization 2: Cache Top Queries at Each Node

Instead of traversing the entire subtree every time, cache the top k queries directly at each node.

Example:

For prefix "be":

Cached Queries
best : 35
bet : 29
bee : 20
be : 15
beer : 10



This optimized trie structure is shown in **Figure**.

Advantages of Caching

Benefits:

- No subtree traversal required
- Extremely fast retrieval
- Constant-time autocomplete lookup

Tradeoff:

- Increased memory usage

This is acceptable because:

- Fast response time is critical
- k is small (usually 5)

Optimized Time Complexity

After optimizations:

Step 1: Find Prefix

None

$O(1)$

Step 2: Return Cached Top k

None

$O(1)$

Final Complexity

Total autocomplete lookup time:

None

$O(1)$

This makes the autocomplete system extremely fast and scalable.

Summary

Technique	Purpose
-----------	---------

Trie	Efficient prefix matching
Frequency storage	Popularity ranking
Prefix length limit	Faster lookup
Cache top queries	Avoid subtree traversal
Cached top k queries	Constant-time response

The optimized trie structure is the core component that enables fast autocomplete suggestions in large-scale systems.